

Bringing the AI-Drafted Reply Digest to X

A development guide for Divinegift — built in Cursor or VS Code, coached by Claude

Written July 2026, as a companion to the Bluesky digest already running on vAI's platform (api/bluesky-digest.ts).

1. Is This Possible?

Yes — the shape of the project is identical to what's already running for Bluesky: search for posts about AI in education in Africa, have Claude draft a reply in vAI's voice for each of the best ones, and email you a digest to review and post from. You'd be building the X version of exactly that pattern.

One thing is genuinely different, and it changes how this project should start: **Bluesky's API is free, but X's is not.** Since February 2026, X moved its developer API to a pay-per-use model with no free tier. Every search costs a small amount of money, billed against credits purchased in advance.

COST REALITY CHECK

Roughly \$0.005 per post read, no monthly minimum. A digest run searching 10 queries × 25 results (matching the Bluesky script's defaults) would cost about **\$1.25 per day (~\$38/month)** if run daily at full size. Starting smaller — 3–4 queries, 10 results each — brings that down to roughly **\$6/month**, which is a sane place to begin while you're still testing.

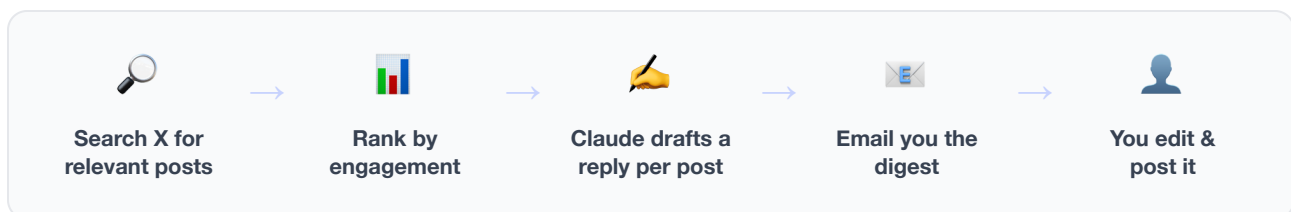
This means step one isn't code — it's a short conversation with Kevin about budget, and someone with billing authority needs to set up the account before you start building. See the next section for exactly who does what.

2. Who Does What

Task	Kevin	Divinegift
X Developer account & billing	Creates the account at <code>console.x.com</code> , adds a payment method, and sets a spending cap in the dashboard.	—
Bearer Token	Generates it in the X developer console, shares it with you as a value to paste into an environment variable — never into chat with Claude, never committed to git.	Uses it locally in a <code>.env.local</code> file that git ignores.
X account to post from	Confirms which X handle replies should go out from (same account @nextvillage.community uses on Bluesky, or a separate one — Kevin's call).	—
Building the script	Available to unblock or review.	Writes, tests, and deploys it — this is your project.
Deploying to production	Reviews and approves the pull request before it merges.	Opens the PR once it works locally. Same process as every other change on this platform — see <code>docs/PR-Process-Guide.pdf</code> if any of this is new.

3. What You're Building

Same five stages as the Bluesky version, just pointed at a different platform:



The reference implementation for all of this already exists at `api/bluesky-digest.ts` in the nextvillage repo. You are not starting from a blank page — you're adapting a working script to a different search API and a tighter character limit. Read that file first, before writing anything new.

4. Environment Setup

- ☐ Install **Cursor** (recommended — it has Claude built in) or **VS Code** with the **Claude Code** extension installed.
- ☐ Install **Node.js** (v20 or newer). Check with `node --version` in a terminal.
- ☐ Get added as a collaborator on the **nextvillage GitHub repo** (ask Kevin), then clone it:

```
git clone https://github.com/khallinan12345/nextvillage.git
cd nextvillage
npm install
```

- ☐ Open the folder in Cursor/VS Code, and open `api/bluesky-digest.ts` — this is your template.
- ☐ Create a `.env.local` file in the project root (it's already git-ignored — never remove it from `.gitignore`) and add placeholders for the new variables you'll need: `X_BEARER_TOKEN`, plus everything `bluesky-digest.ts` already uses (`ANTHROPIC_API_KEY`, `RESEND_API_KEY`, etc. — ask Kevin to share these once, the same way he'll share the X token).
- ☐ Get added as a collaborator on the **Vercel project** (`girls-aiing-and-vibing`) so you can set environment variables and see deploy logs yourself later.

5. Working With Claude on This — Ground Rules

This platform's whole philosophy is AI that coaches instead of answers. Building this the same way is the point, not a nice-to-have. A few rules that make that actually happen instead of just becoming "paste this whole file and hope":

- **Build one small piece at a time.** Don't ask Claude to generate the whole script in one shot. Get the search working and tested before touching drafting; get drafting working before touching email.
- **Ask Claude to explain, not just generate.** When you get code back, ask "explain what this does line by line" before moving on. If you can't explain a piece back in your own words, you're not done with that step yet.
- **Never paste real credentials into a Claude chat.** Not the Bearer Token, not API keys, not passwords. Use placeholder values when asking for help, and only put real values directly into `.env.local` or Vercel's environment variable settings.
- **Test after every step**, before moving to the next one. Run the script locally and look at what it actually printed — don't assume it worked because Claude said it should.
- **When something breaks, paste the actual error message** into the chat, not a description of the error. Claude can diagnose an exact stack trace far better than a paraphrase.

6. The vAI Voice (Reused From Bluesky, Adjusted for X)

Reply drafts should follow the same two-beat structure as the Bluesky version: **briefly and genuinely lift up the original post first**, then **lift up whichever one of vAI's three unique elements fits best** — don't force all three into every reply.

1. **Community ownership & leadership** — the community owns and leads its own AI Learning and Tech Skills platform (nextvillage.community). Local young people are the developers, facilitators, and the platform's voice.
2. **Human-centered process** — everything starts from listening to the community's own aspirations, shaped by decades of engineering-service-learning practice (ETHOS, founded 2001).
3. **AI that coaches instead of answers** — the platform's AI develops learners' thinking rather than replacing it; certification requires demonstrated reasoning, not copied output.

THE ONE REAL DIFFERENCE FROM BLUESKY

X's post limit is **280 characters**, not 300. When you adapt the drafting prompt from `bluesky-digest.ts`, change the character instruction accordingly — and still leave a little headroom, the same way the Bluesky version targets 290 of its 300 to survive the model slightly miscounting.

7. Step-by-Step Build Plan

1 Understand the reference file

PROMPT TO GIVE CLAUDE

```
Explain what api/bluesky-digest.ts does, section by section, as if I'm new to this codebase. I'm about to adapt it to search X instead of Bluesky — call out anything Bluesky-specific I'll need to change.
```

2 Confirm the current X API details before writing code

API terms shift — this guide was written July 2026. Before building, verify against X's live docs rather than trusting older training data (yours or Claude's).

PROMPT TO GIVE CLAUDE

```
Look up X's current API docs at docs.x.com. I need the exact endpoint for searching recent posts (last 7 days) by keyword, the auth header format for a Bearer Token, and current pay-per-use pricing. Confirm these match what I have before we write anything.
```

3 Write and test the search function, alone

Mirror `searchBluesky()` in the reference file, but authenticated with the Bearer Token from the start (X has no unauthenticated path at all — don't repeat the mistake the Bluesky version originally made). Log the results to the console. Don't touch drafting or email yet.

PROMPT TO GIVE CLAUDE

```
Help me write a searchX(query) function modeled on searchBluesky() in this file, but using X's search endpoint with Bearer Token auth. I want to run it standalone and just console.log the results first.
```

4 Rank and dedupe — this part barely changes

The ranking/scoring logic in `collectPosts()` is not platform-specific. Reuse it almost as-is, adjusted for whatever field names X's API returns for likes/reposts/replies.

5 Adapt the drafting prompt

Copy `DRAFT_SYSTEM_PROMPT` from the reference file, change the character limit to 280 (aim for 270), and keep everything else — the voice, the structure, the Oloibiri context — as is. Same Anthropic API call pattern, same JSON-array-of-drafts output format.

6 Wire up the email

Reuse `formatEmail()` and `sendDigest()` almost unchanged — same Resend setup already configured for this project. Just relabel "Bluesky" to "X" in the copy.

7 Add the Vercel handler — watch this specific trap

A REAL BUG FROM BUILDING THE BLUESKY VERSION

It's tempting to write `export async function GET(request: Request)` — that's a Next.js convention and it does **not** work in this project. Every function here uses:

```
export default async function handler(req: VercelRequest, res: VercelResponse) {
  const cronSecret = process.env.CRON_SECRET;
  if (cronSecret && req.headers["authorization"] !== `Bearer ${cronSecret}`) {
    return res.status(401).json({ error: "Unauthorized" });
  }
  // ...
}
```

Copy that pattern exactly. If you get this wrong, the function deploys fine and just silently never runs on schedule — one of the harder bugs to notice.

8 Set environment variables the right way

ANOTHER REAL TRAP, FOUND THIS WEEK

Piping a value into `vercel env add NAME production` from the command line can report success while silently storing an **empty** value — this happened with this project's own `DIGEST_RECIPIENTS` variable. Use the explicit flag instead, and always verify:

```
vercel env add X_BEARER_TOKEN production --value "<token>" --force --yes

# then verify it actually took:
vercel env pull .env.check --environment production
grep X_BEARER_TOKEN .env.check
rm .env.check
```

9 Full local dry run before deploying

Run the whole pipeline locally with real credentials, end to end, and confirm you actually receive a digest email before touching Vercel at all:

```
npx tsx --env-file=.env.local scratch-driver.mts
```

(Ask Claude to help you write a small driver script that calls the handler directly with a fake request/response — the same technique used to test the Bluesky version.)

10 Open a PR — don't merge it alone

Same process as everything else on this platform: branch, commit, push, open a pull request against `main`, and get Kevin to review before it merges. Add the new cron entry to `vercel.json` in the same PR.

8. Cost & Safety Guardrails

- Start with 3–4 search queries and a small results limit (10, not 25) while testing. Widen it later once you trust the output.
- Set a spend alert in the X developer console so a bug can't run up a large bill unattended.
- Keep the digest as a **daily** cron, not more frequent — matches Bluesky's cadence and keeps cost predictable.
- Drafts are a starting point, never an auto-post. Nothing should go out under your name without you reading it first — same rule as Bluesky.

9. Glossary

Term	Means
Bearer Token	A credential string sent in a request header (<code>Authorization: Bearer <token></code>) that proves to X's API who's asking. Functions like a password, but scoped to API access only.
Rate limit	A cap on how many requests you can make in a given time window, enforced by X regardless of whether you've paid for credits.
Pay-per-use credit	Prepaid balance that X deducts from as your app reads or posts. No credits left means requests start failing.
Cron job	A scheduled task that runs automatically on a timer — Vercel triggers this project's digest once a day without anyone clicking anything.
Env var (environment variable)	A named secret or setting (like an API key) stored outside the code itself, so it never gets committed to git or shown in the source.
Endpoint	A specific URL path an API exposes for one purpose — e.g. "the endpoint for searching recent posts."

10. When You're Stuck

- Paste the exact error message into Claude, along with what step you were on and what you expected to happen instead.
- If a step in this guide doesn't match what you're actually seeing (an API has changed, a file has moved), say so explicitly — ask Claude to verify current behavior rather than assuming this document is still accurate. It won't age perfectly.
- If you're blocked on something only Kevin can unblock — budget, credentials, repo access — say that plainly rather than working around it.